

第一章 概述

知识点名称	内容
计算机发展 ★	1. 第一代：20 世纪 50 年代，主要采用真空电子管制造计算机。 2. 第二代：20 世纪 50 年代末期，出现了以晶体管为主要元件的第二代计算机。 3. 第三代：1964 年，IBM 公司推出 IBM 360 系列计算机 4. 第四代：随着大规模集成电路和微处理器的出现，计算机进入第四代。
计算机语言 ★★★★	1. 机器语言：用二进制代码表示的能被计算机识别和执行的指令集合。 2. 汇编语言：机器语言和汇编语言都是面向机器的语言——“低级语言”。 3. 高级语言：翻译程序的工作方式：编译方式和解释方式。世界上第一个高级语言是 FORTRAN，后来陆续出现很多种高级语言，目前广泛应用的高级语言有 C/C++、Java 和 C#等。 (1) 编译方式：将源程序全部翻译为二进制程序后再执行，在此完成翻译工作的程序称为“编译程序”，编译后的二进制程序称为“目标程序”； (2) 解释方式：边解释边执行，完成翻译工作的程序称为“解释程序”。
算法及其描述方法 ★	1. 算法：解决问题的步骤序。特征：可执行性、确定性、有穷性、有输入信息的说明、有输出信息的步骤。 2. 算法描述方法：自然语言、传统流程图、N-S 流程图（三种基本结构：顺序结构、选择结构、循环结构）、伪代码、计算机语言。
程序和程序设计方法 ★★	1. 程序=数据结构+算法 2. 计算机程序：指根据算法描述，用计算机语言表示的能被计算机识别和执行的指令集合。 3. 程序设计方法： (1) 结构化程序设计方法（自顶向下、逐步求精），程序由三种基本结构构成：顺序、选择和循环。 (2) 面向对象程序设计方法（由类、对象和对象之间的动态联系组成）。 (3) 二者之间的最大差别表现在： 1) 面向对象方法采用数据抽象和信息隐藏技术使组成类的数据和操作是不可分割的，避免了结构化程序由于数据和过程分离引起的弊病。 2) 面向对象的程序是由类、对象（类的实例）和对象之间的动态联系组成的。而结构化程序是由结构化的数据、过程的定义以及调用过程处理相应的数据组成的。

第二章 C 语言基础知识

知识点名称	内容
C 语言特点 ★	(1) 结构化语言 (2) 运算能力强大 (3) 数据类型丰富 (4) 具有预处理能力 (5) 可移植性好 (6) 程序执行效率高 (7) 程序设计自由度高
C 语言基本词法 ★★★★	1. 字符是组成语言最基本的元素。 2. C 语言的字符集由字母、数字、空格、标点和特殊字符组成。ASCII 字符集包括： (1) 阿拉伯数字：0、1、2、…、9(共 10 个) (2) 大小写英文字母：A~Z、a~z(共 52 个) (3) 非字母、非数字的可显示字符（33 个） (4) 转义字符：由反斜杠“\”后面跟单个字符或若干个字符组成

	2. 关键	<table><tr><td>关键字</td><td>含义</td><td>关键字</td><td>含义</td><td>关键字</td><td>含义</td></tr><tr><td>auto</td><td>自动</td><td>extern</td><td>外部</td><td>sizeof</td><td>计算字节数</td></tr><tr><td>break</td><td>中止</td><td>float</td><td>浮点</td><td>static</td><td>静态</td></tr><tr><td>case</td><td>情况</td><td>for</td><td>对于</td><td>struct</td><td>结构</td></tr><tr><td>char</td><td>字符</td><td>goto</td><td>转向</td><td>switch</td><td>开关</td></tr><tr><td>continue</td><td>继续</td><td>int</td><td>整</td><td>typedef</td><td>类型定义</td></tr><tr><td>const</td><td>常量</td><td>if</td><td>如果</td><td>union</td><td>共用</td></tr><tr><td>default</td><td>默认</td><td>long</td><td>长</td><td>unsigned</td><td>无符号</td></tr><tr><td>do</td><td>做</td><td>register</td><td>寄存器</td><td>void</td><td>空</td></tr><tr><td>double</td><td>双</td><td>return</td><td>返回</td><td>volatile</td><td>可变的</td></tr><tr><td>else</td><td>否则</td><td>short</td><td>短</td><td>while</td><td>当</td></tr><tr><td>enum</td><td>枚举</td><td>signed</td><td>有符号</td><td></td><td></td></tr></table>	关键字	含义	关键字	含义	关键字	含义	auto	自动	extern	外部	sizeof	计算字节数	break	中止	float	浮点	static	静态	case	情况	for	对于	struct	结构	char	字符	goto	转向	switch	开关	continue	继续	int	整	typedef	类型定义	const	常量	if	如果	union	共用	default	默认	long	长	unsigned	无符号	do	做	register	寄存器	void	空	double	双	return	返回	volatile	可变的	else	否则	short	短	while	当	enum	枚举	signed	有符号		
关键字	含义	关键字	含义	关键字	含义																																																																					
auto	自动	extern	外部	sizeof	计算字节数																																																																					
break	中止	float	浮点	static	静态																																																																					
case	情况	for	对于	struct	结构																																																																					
char	字符	goto	转向	switch	开关																																																																					
continue	继续	int	整	typedef	类型定义																																																																					
const	常量	if	如果	union	共用																																																																					
default	默认	long	长	unsigned	无符号																																																																					
do	做	register	寄存器	void	空																																																																					
double	双	return	返回	volatile	可变的																																																																					
else	否则	short	短	while	当																																																																					
enum	枚举	signed	有符号																																																																							
	3. 标识	1. C 语言规定，用户标识符仅由大小写英文字母、数字和下画线组成，且第一个字符不能是数字。 2. 用户自定义的标识符既不能是 C 语言的关键字，也不能与用户已编写的函数或 C 语言的库函数重名。																																																																								
C 程序 基本组成 ★★★★	1. C 程序的组成特点如下： (1) 每个 C 程序由一个或多个函数组成。每个 C 程序有且仅有一个主函数，其函数名为 main。 (2) 函数是 C 程序的基本单位。 (3) C 程序中每一语句后面都以分号“;”作为语句的结束。 (4) C 语言本身没有输入/输出语句，而是通过调用库函数 <code>scanf()</code>、<code>printf()</code>、<code>getchar()</code> 和 <code>putchar()</code> 实现的。 (5) 多行注释/*.....*/。单行注释加“//”即可。 (6) C 程序的执行总是从主函数开始，并在主函数结束。主函数和其他函数的位置是任意的，其他函数总是通过函数调用语句被执行。 (7) 主函数可以调用任何其他函数，任何其他函数之间可以互相调用，但不能调用主函数。																																																																									
C 程序 开发环境 ★	1. C 程序的运行步骤：(1) 编辑：建立源程序文件扩展名为“c”或“cpp”。(2) 编译：编译程序将源程序转换为二进制形式的目标程序，扩展名为“obj”。(3) 链接：生成可执行文件，扩展名为“exe”。 (4) 运行																																																																									

第三章 数据类型、运算符和表达式

知识点名称	内容
数据类型★	1. 数据类型包括：基本类型（整型、实型、字符型和枚举型）、构造类型、指针类型和空类型。
常量 ★★★★	1. 常量 又称为常数，是在程序运行过程中其值不能改变的数据。
	2. 整型常量 (1) 八进制 整型常量（必须以 0 开头，0-7 组成）、 (2) 十进制 整型常量（通常的整数）、 (3) 十六进制 整型常量（以 0X 或 0x 开头，0-9、a-f 或 A-F 组成）
	3. 实型常量 实型常量又称浮点型常量或实数，只使用十进制。两种书写形式： (1) 十进制小数形式（必须有小数点）。

		(2) 指数形式 (aEb 或 aeb, a 为尾数部分, b 为指数部分且必须为整数, 均有正负之分)																																						
	4. 字符常量	包括普通字符常量 ('a') 和转义符常量 (如 '\\', '\n')。使用一对单引号括起来的单个字符。在内存中占用 1 字节, 具体存放的是该字符对应的 ASCII 码值。																																						
	5. 字符串常量	字符串常量又称"字符串", 使用一对双引号括起来。除了要存储有效的字符外, 还要存储一个字符串结束标志 '\0'。长度为 n 个字符的字符串占用 n+1 字节。																																						
	6. 符号常量	符合常量定义格式为 #define 符号常量 常量																																						
变量 ★★★★	变量的数据类型	1. 整型变量 (1) 基本整型: 类型关键字为 int。 (2) 短整型: 类型关键字为 short[int]。 (3) 长整型: 类型关键字为 long[int]。 (4) 无符号整型: 无符号基本整型 (unsigned [int])、无符号短整型 (unsigned short) 和无符号长整型 (unsigned long) 三种。 ①16 位编译器中: long 型 (4 字节) ≥int 型 (2 字节) ≥short 型 (2 字节); ②32 位编译器中: 一个 int 型变量占用 4 字节, long 型变量占用 4 字节, short 型变量占用 2 字节。 2. 实型变量 (1) 单精度实型, 类型关键字为 float, 一般占用内存 4 字节。 (2) 双精度实型, 类型关键字为 double, 一般占用内存 8 字节。 3. 字符变量: 关键字是 char, 存放的是字符的 ASCII 码值, 占用内存 1 字节。																																						
	变量的定义和初始化	4. 变量定义格式: 数据类型符 变量名 1[, 变量名 2, ...]; 5. 变量初始化格式: 数据类型符 变量名 1=初值 1[, 变量名 2[=初值 2], ...];																																						
运算符和表达式 ★★★★★★	1. 优先级和结合性	<table><tr><td>优 先 级</td><td>运 算 符</td><td>结 合 性</td></tr><tr><td>1</td><td>() [] -> .</td><td>从左至右</td></tr><tr><td>2</td><td>! ~ ++ -- * (取内容) & (取地址) + (取正) - (取负) sizeof</td><td>从右至左</td></tr><tr><td>3</td><td>* (乘) / %</td><td rowspan="8">从左至右</td></tr><tr><td>4</td><td>+ (加) - (减)</td></tr><tr><td>5</td><td><< >></td></tr><tr><td>6</td><td>< <= > >=</td></tr><tr><td>7</td><td>== !=</td></tr><tr><td>8</td><td>& (位逻辑与)</td></tr><tr><td>9</td><td>^</td></tr><tr><td>10</td><td> </td></tr><tr><td>11</td><td>&&</td></tr><tr><td>12</td><td> </td></tr><tr><td>13</td><td>?:</td><td rowspan="2">从右至左</td></tr><tr><td>14</td><td>= += -= *= /= %= &= ^= = >>= <<=</td></tr><tr><td>15</td><td>,</td><td>从左至右</td></tr></table> 注: 优先级的数字越小, 级别越高。	优 先 级	运 算 符	结 合 性	1	() [] -> .	从左至右	2	! ~ ++ -- * (取内容) & (取地址) + (取正) - (取负) sizeof	从右至左	3	* (乘) / %	从左至右	4	+ (加) - (减)	5	<< >>	6	< <= > >=	7	== !=	8	& (位逻辑与)	9	^	10		11	&&	12		13	?:	从右至左	14	= += -= *= /= %= &= ^= = >>= <<=	15	,	从左至右
		优 先 级	运 算 符	结 合 性																																				
		1	() [] -> .	从左至右																																				
		2	! ~ ++ -- * (取内容) & (取地址) + (取正) - (取负) sizeof	从右至左																																				
		3	* (乘) / %	从左至右																																				
		4	+ (加) - (减)																																					
		5	<< >>																																					
		6	< <= > >=																																					
		7	== !=																																					
		8	& (位逻辑与)																																					
		9	^																																					
		10																																						
		11	&&																																					
		12																																						
		13	?:	从右至左																																				
	14	= += -= *= /= %= &= ^= = >>= <<=																																						
15	,	从左至右																																						
2. 算术运算符	算术运算符包括+、-、*、/、+(取正)、-(取负), 自增运算符++、自减运算符--(单																																							

		目前缀后引用，单目后缀先引用），算术表达式（由运算对象、圆括号和算法运算符组成）
	3. 关系运算符	关系运算符包括>、>=、<、<=、==、!=，逻辑结果值为真（1）或假（0）
	4. 逻辑运算符	逻辑运算符包括&&、 和!三种，用于对关系式或逻辑值（真、假）进行运算，运算结果是逻辑值
	5. 赋值运算符	赋值运算符分为基本赋值运算符（=）、复合赋值运算符（+=、-=、*=、/=、%=、&=、^=、 =、<<=、>>=）
	6. 逗号运算符	逗号运算符又称顺序求值运算符、它是双目运算符、运算对象是表达式，常用于 for 循环语句中
	7. 条件运算符	条件运算符格式：表达式1? 表达式2: 表达式3，执行过程表达式1为真，返回表达式2，若为假返回表达式3
	8. 位运算符	位运算符分为位逻辑运算符（按位非~、按位与&、按位或 、按位异或^）和位移运算符（左移<<、右移>>）
	9. 长度运算符	sizeof(数据类型符)或 sizeof(变量)
	数据类型转换 ★★	1. 自动类型转换：参与运算的各个数据都转换成数据较长的数据类型，结果的类型就是数据长度较长的数据类型。即“就高不就低”或“就长不就短”的原则。 2. 赋值类型转换：先将运算结果的数据类型自动转换为左边变量的数据类型，然后再赋予该变量。本质上是“就左不就右”的转换原则。 3. 强制类型转换：格式：(数据类型符)(表达式)。注意：若表达式仅仅是单个常量或变量时，外面的一对圆括号可以省略；若是含有运算符的表达式，则必须用一对圆括号括起来。

第四章 结构化程序设计

知识点名称	内容	
结构化程序设计方法★	结构化编码：经模块化设计后进入编码阶段，任何程序都由顺序、选择和循环三种基本结构组成。	
顺序结构设计 ★★★★★	1. 赋值语句	赋值语句的格式是： (1) 变量=表达式； (2) 变量 复合赋值运算符 表达式；
	2. 字符输入/输出函数	(1) 输出函数 putchar(ch) 【功能】把一个字符输出到标准输出设备（显示器）。 (2) 输入函数 getchar() 【功能】从标准输入设备（键盘）输入一个字符。
	3. 格式输入/输出函数	输出函数 printf(格式控制字符串, 输出项表) 【功能】按照用户指定的格式把指定的数据输出到标准输出设备（显示器）。 (1) 格式说明符：由“%”和“格式字符”组成。 输出不同类型的数据，需要使用不同的格式字符，其中： 1) 格式字符 d——以有符号的十进制整数形式输出。 2) 格式字符 f——以小数形式、按系统默认的宽度，输出单精度和双精度实数。 3) 格式字符 c——输出一个字符（只占一列宽度）。

		4) 格式字符 s——输出一个字符串。 (2) 常用的输出格式 1) 有符号整型数据: %d 2) 有符号长整型数据: %ld 3) 实型数据: %m.nf 4) 字符串数据: %s 5) 无符号整型数据: %u 6) 无符号长整型数据: %lu 7) 字符型数据: %c	
		输入函数 scanf (格式控制字符串, 输入项首地址表) 【功能】从键盘按照“格式控制字符串”中规定的格式读取若干个数据, 然后按照“输入项首地址表”中的顺序, 依次将数据存入相应的变量对应的地址。 (1) 格式字符 1) d——以有符号的十进制整数形式输入。 2) o——以无符号的八进制整数形式输入。 3) x——以无符号的十六进制整数形式输入。 4) c——输入一个字符。 5) f、e——输入实型数据 6) s——输入一个字符串。 (2) 常用的输入格式 1) 有符号整型数据: %d 2) 有符号长整型数据: %ld 3) 单精度实型数据: %f 4) 双精度实型数据: %lf 5) 无符号整型数据: %u 6) 无符号长整型数据: %lu 7) 字符型数据: %c 8) 字符串数据: %s	
选择结构程序设计 ★★★★★	1. 单分支选择语句	【格式】if (表达式) 语句; 【功能】计算表达式的值, 若为“真”(非 0), 则执行语句; 否则不执行语句。	
	2. 双分支选择语句	【格式】if (表达式) 语句 1; else 语句 2; 【功能】计算表达式的值, 若为“真”(非 0), 则执行语句 1; 否则执行语句 2。	
	3. 分支的嵌套	(1) if 语句的嵌套	<pre> if (表达式 1) if (表达式 2) 语句 1; else 语句 2; else if (表达式 3) 语句 3; </pre>

			else 语句 4;
		(2) if-else-if 语句的嵌套	if (表达式 1) 语句 1 else if (表达式 2) 语句 2 else if (表达式 3) 语句 3 ... else if (表达式 n) 语句 n else 语句 n+1
	4. 多分支选择语句	switch(表达式) { case 常量表达式 1:语句组 1;break; case 常量表达式 2:语句组 2;break; ... case 常量表达式 n:语句组 n;break; [default:语句组 n+1;[break;]] }	
循环结构 程序设计 ★★★	1. while 语句 要用来实现当型循环结构。 (1) 格式: while(表达式) 语句; (2) 功能: 计算表达式值, 为真则执行语句, 重复执行上述操作直至表达式值为假时止 2. do-while 语句 主要用来实现直到型循环结构。 (1) 格式: do 语句; while(表达式); (2) 功能: 执行语句, 计算表达式的值, 为假退出循环, 为真继续循环 3. for 语句 主要用来实现次数型循环结构。 (1) 格式: for(表达式 1; 表达式 2; 表达式 3) 语句; (2) 功能: 第一步, 计算表达式 1, 第二步计算表达式 2, 若为假退出循环, 否则执行语句, 再计算表达式 3, 转第二步 4. continue 语句 (1) break 语句, 强制结束当前的循环, 不再执行循环体中 break 后面的语句, (2) continue 语句, 跳过 continue 之后的语句, 重新判断循环控制条件, 决定是否继续循环		

第五章 数组

知识点名称	内容
一维数组 ★★★★★	定义: 数据类型符 数组名 1[长度 1], 数组名 2[长度 2], ...; 例: int a[4]; (数组同变量一样, 必须先定义后使用) 数组元素的下标是从 0 开始顺序编号。 元素的引用: 数组名[下标] “下标” 可以使整型常量、整型变量或整型表达式, 其取值范围是 0~长度-1 例: a[0]=1; 初始化: 数据类型符 数组名[长度]={初值表}, ...; 注意: 数组可全部或部分赋值, 赋值格式不得超过长度, 全部赋值可省略长度; 如果被定义数组的长度, 与初值个数不同, 则数组长度不能省略。

二维数组 ★★★★	1. 定义：数据类型符 数组名[行长度][列长度]，...； 例：int a[3][4]； 2. 引用：数组名[行下标][列下标] 3. 初始化一般形式：数据类型符 数组名[行长度][列长度]={第 0 行初值表}，{第 1 行初值表}，... {最后 1 行初值表}；
字符数组和字符串★	字符串是由若干个字符组成的，其最后一个字符是字符串的结束标志符“'\0'”。故数组长度是字符串的字符个数加 1。
字符数组 ★★★★	1. 一维字符数组的定义形式：char 数组名[数组长度]={初值表}； 2. 二维字符数组定义形式：char 数组名[行长度][列长度]={初值表}，{初值表}，...，{初值表}；
字符串 ★★★★	1. 字符串是用双引号括起来的一串字符，在字符串末尾系统会自动地添加一个字符串结束标志符'\0'。 2. 字符串的输入和输出： (1) 使用 scanf() 和 printf() 函数 1) 逐个字符输入和输出。在 scanf() 函数或 printf() 函数中用“%c”格式说明符每次输入或输出一个字符，重复该过程完成整个字符串的输入和输出。 2) 字符串整个输入和输出。在 scanf() 函数或 printf() 函数中用“%s”格式说明符一次性输入或输出整个字符串。 (2) 使用 gets() 和 puts() 函数。使用字符串输入函数 gets() 或字符串输出函数 puts() 将字符串整个输入或输出。
字符串处理的常用库函数 ★★★★	1. 字符串输入函数 gets() 【调用格式】 gets(字符数组名) 【功能】 从标准输入设备即键盘上输入一个字符串（可以包含空格，仅以回车作为结束标志），并将其存储到指定的字符数组中。 2. 字符串比较函数 strcmp() 【调用格式】 strcmp(字符串 1, 字符串 2) 【功能】 比较两个字符串的大小，即两个字符串从左到右逐个字符进行比较（按照 ASCII 码值的大小进行比较），直到出现不同字符或遇到 '\0' 为止。 3. 字符串复制函数 strcpy() 【调用格式】 strcpy(字符数组名, 字符串[, 整型表达式]) 【功能】 将“字符串”的前“整型表达式”个字符组成新的字符串复制到“字符数组”中，若省略“整型表达式”，则将“字符串”完整地复制到“字符数组中”，字符数组的原有内容被覆盖。 4. 字符串连接函数 strcat() 【调用格式】 strcat(字符数组名, 字符串) 【功能】 把“字符串”连接到“字符数组”中字符串的尾端（最后一个有效字符的后面），组成新的字符串并存储到“字符数组”。“字符数组”中原来的结束标志，被“字符串”的第一个字符覆盖，而“字符串”在操作中未被修改。 5. 测字符串长度函数 strlen() (其中 len 是 length 的缩写) 【调用格式】 strlen(字符串) 【功能】 测字符串（字符串常量或字符数组）的实际长度（不包含字符串结束标志符 '\0'）。

第六章 函数

知识点名称	内容
语言函数的特点 ★★★	1. 一个 C 程序由一个或多个函数组成，其中必须有且只能有一个 main 函数（称为主函数）。 2. C 程序的执行从主函数开始，如果在主函数中调用其他函数，调用后返回到主函数，并在主函数中结束整个程序的运行。 3. 主函数可以调用其他函数，但其他函数不能调用主函数。主函数由操作系统调用，其他函数之间可以互相调用
函数的定义 ★★★★★	1. 一般格式： 数据类型符 函数名(形式参数表) { 数据定义语句部分; 执行语句部分; } 2. 各类函数定义的格式 有返回值函数和无返回值函数的主要区别体现在两个方面。一是数据类型符，有返回值函数的数据类型符可以选取任意的数据类型符，而无返回值函数的数据类型符只能是 void；二是有返回值函数的函数体中必须有语句 return(表达式)；而无返回值函数的函数体中的语句 return；可以有也可以省略。 3. 返回值类型：即函数类型，是指返回给主调函数的结果的类型，应根据具体函数的功能确定。如果函数不返回任何值，则函数返回值类型定义为 void，称为“无类型”。
函数的调用 ★★	1. 函数调用的一般形式：函数名([实际参数表])； 2. 函数调用的方式：（1）以函数表达式的方式调用函数。（2）以函数语句的方式调用函数。（3）以函数实参的方式调用函数。 3. 函数的形式参数和实际参数 （1）形参是在函数定义时设置的，用来接收从主调函数传来的对应的实参数据。 （2）实参是调用函数时的实际参数，常量、变量、表达式、函数的返回值都可，必须有确定的值。 实参在数量、类型和顺序上与形参必须一一对应和匹配。 4. 返回语句有以下两种使用方式： （1）无返回值的函数：return； （2）有返回值的函数：return(返回值表达式)；或 return 返回值表达式；
函数声明 ★★	1. 库函数，除了少数库函数（如 scanf()、printf()）外，都要求在本文件开头用文件包含命令 #include<头文件名.h>或#include"头文件名.h"包含被调函数的信息。 2. 用户函数，若主调函数与被调函数不在同一个编译单元，则在定义函数的编译单元中必须将该函数定义为外部函数，同时主调函数的函数体或主调函数所在编译单元的开头将被调函数按照如下格式进行声明：extern 数据类型符 函数名(形参表)；

函数的参数和数据传递方式 ★★★★★	1. C语言规定在函数之间传递数据包括值传递、地址传递、返回值和全局变量传递四种方式。 2. 值传递方式：在形参和实参间传递数据的一种方式，值传递方式传递的是参数值。判断是否是值传递方式的唯一方法是看函数定义时的形参是不是变量形式。如果形参是变量，则是值传递方式。 3. 数组作为函数参数的数据传递方式 (1) 数组元素作为函数实参：数组元素即下标变量，它与普通变量没有区别。数组元素只能用做函数实参，和变量作为函数实参一样。在调用函数时，把数组元素的值传递给形参，实现单向的值传递方式。 (2) 数组名作为函数参数：数组名作为函数参数时，既可以作为形参，也可以作为实参。如果形参是数组名，则传递方式称为“地址传递方式”。调用函数时的实参是地址型表达式，例如数组的首地址或已经赋值的指针变量、指针数组元素等。
变量的存储类型 ★★★★★	1. C语言中变量的存储类型分为自动型 (auto)、寄存器型 (register)、外部型 (extern)和静态型 (static)四种。 (1) 自动型变量 <u>[auto] 数据类型 变量表;</u> 关键字 auto 可以省略，C语言默认 auto 型 (2) 静态型变量 <u>static 数据类型 变量表;</u> 【说明】静态变量只在本文件中起作用。 (3) 寄存器型变量 register 数据类型 变量表; 只有局部变量才能定义为寄存器型变量 (4) 外部型变量 extern 数据类型 变量表; 专门用于在多个编译单元之间传递数据的。
变量的生存期和作用域 ★★	1. 全局变量和局部变量 (1) 生存期覆盖了定义点到整个程序结束的变量称为“全局变量”； (2) 生存期只覆盖了某个函数（或复合语句）的变量称为“局部变量”。 (3) 定义全局变量时如果没有初始化，则自动赋值 0（整型和实型）或‘\0’（字符型）。用于说明全局变量的存储类型的关键字是 extern 和 static。 2. 内部变量和外部变量 (1) 内部变量的作用域：主函数中定义的内部变量，只能在主函数中使用，其他函数不能使用。 (2) 外部变量，需要注意： 1) 在同一源程序文件中，允许外部变量和内部变量重名。如果外部变量和内部变量重名，则在内部变量的作用域内，外部变量将被屏蔽而不起作用。 2) 外部变量的作用域是从定义点到本文件结束。如果定义点之前的函数需要使用这些外部变量时，需要在函数内对被使用的外部变量进行声明，格式：extern 数据类型 外部变量 1[, 外部变量 2, ...];
函数的嵌套调用 ★★★★★	1. 函数的嵌套调用是指在调用函数时，被调函数又调用了其他函数或自身。当被调函数是函数自身时，称为函数的递归调用，递归调用是嵌套调用的一种特例。
函数的递归调用 ★★★★★	1. 在调用一个函数的过程中又直接或间接地调用其自身，称递归调用。递归调用又称自调用函数。 (1) 如果函数 sum() 调用函数 sum() 自身，称为直接递归调用。 (2) 如果函数 sum1() 调用函数 sum2()，函数 sum2() 中又调用函数 sum1()，称为间接递归调用。
常用的数学处理函数 ★★★★	1. 计算长整型绝对值函数：long labs(long x) 2. 计算实型绝对值函数：int abs(int x) 【返回值】返回整数 x 的绝对值。 3. 计算小于或等于 x 的最大整数函数：double floor(double x) 4. 计算大于或等于 x 的最小整数函数：double ceil(double x)

	5. 计算平方根函数 : <code>double sqrt(double x)</code> 【返回值】返回 <code>x</code> 的平方根。 6. 计算常用对数函数: <code>double log10(double x)</code> 7. 计算自然对数函数: <code>double log(double x)</code> 8. 计算指数函数: <code>double exp(double x)</code> 9. 计算 10 的 <code>x</code> 次幂函数: <code>double pow10(int x)</code> 10. 计算 <code>x</code> 的 <code>y</code> 次方函数 : <code>double pow(double x, double y)</code> 【返回值】返回 <code>x</code> 的 <code>y</code> 次方的值。
--	---

第七章 指针

知识点名称	内容
指针★	指针是存放数据的内存单元地址。
指针变量 ★★★★★	1. 指针变量 是专门用于存储其他变量地址的变量。 2. 指针变量的定义和初始化 <u>数据类型符 *指针变量名 [=初始地址值], ...;</u> 3. 指针变量的一般使用 (1) 给指针变量赋值: 指针变量=地址型表达式 (2) 直接使用指针变量: 指针变量名 (3) 通过指针变量引用所指向的变量: * 指针变量名 4. 指针的基本运算 : (1) 取地址运算符“&” 的功能是取变量的地址, 它是单目运算符。 (2) 指针运算符“*” 的功能是取指针变量所指向地址中的内容, 与取地址运算符“&”的运算是互逆的, 它是单目运算符。
指针和一维数组 ★★★★★	1. 将指针变量指向一维数组的方法 : (1) 在数据定义语句中用赋初值的方式, 即*指针变量=数组名。 (2) 在程序中用赋值的方式, 即指针变量=数组名;。 2. 将指针变量指向一维数组元素的方法 : (1) 在数据定义语句中用赋初值的方式, 即*指针变量=&数组名[下标]。 (2) 在程序中用赋值的方式, 即指针变量=&数组名[下标];。 3. 当指针变量指向一维数组, 利用指针变量引用一维数组元素的方法如下 : (1) 引用下标为 0 的数组元素 *(指针变量+0) 或 *指针变量 或 指针变量[0] (2) 引用下标为 <code>i</code> 的数组元素 *(指针变量+ <code>i</code>) 或 指针变量[<code>i</code>] 4. 当指针变量指向下标为 <code>i</code> 的一维数组元素时, 利用指针变量引用数组元素的方法如下 : (1) 引用下标为 <code>i</code> 的数组元素 *(指针变量+0) 或 *指针变量 (2) 引用下标为 <code>i-k</code> 的数组元素 *(指针变量- <code>k</code>) (3) 引用下标为 <code>i+k</code> 的数组元素 *(指针变量+ <code>k</code>) 5. 当指针变量指向一维数组时, 引用下标为 <code>i</code> 的一维数组元素可以采用四种方法 。 (1) *(指针变量+ <code>i</code>) (2) *(数组名+ <code>i</code>) (3) 指针变量[<code>i</code>] (4) 数组名[<code>i</code>]
指针变量的运算 ★	1. 赋值运算 例如: 设 <code>int i, *p</code> , 将变量 <code>i</code> 的地址赋予指针变量 <code>p</code> 为: <code>p=&i</code> ; 2. 算术运算 (1) 指针变量±整数。

	(2) 指针变量++/++指针变量。 (3) 指针变量--/--指针变量。 (4) 指针变量 1-指针变量 2。 3. 关系运算 两个类型相同的指针变量可以运用关系运算符比较大小，表示两个指针变量所指向地址位置的前后关系，即前者为小，后者为大。
指向字符串的指针变量 ★★★	1. 将指针变量指向字符串的方法： (1) 在数据定义语句中用赋初值的方式：*指针变量=字符串 (2) 在程序中用赋值的方式：指针变量=字符串； 2. 当指针变量指向字符串时，利用指针变量处理字符串的方式： (1) 输出整个字符串 printf("%s", 指针变量)； (2) 输入整个字符串 scanf("%s", 指针变量)； (3) 处理字符串中的单个字符 利用 for 循环和指向字符串的指针变量 string 逐一输出字符串字符。
指向字符数组的指针变量★	1. 字符串是按字符数组进行处理的，系统存储一个字符串时先分配一个起始地址，从该地址开始连续存放字符串中的字符。这一起始地址即是字符串首字符的地址。 2. 可以将一个字符串赋值给一个字符数组，也可以赋值给一个字符指针变量。
字符指针和字符数组的区别 ★★	1. 存储内容不同。字符数组在定义后，编译系统为其分配一段连续的存储单元，这段内存单元的起始地址可以用数组名表示，其中存储的是字符串本身；而字符指针变量在定义后，编译系统仅为其分配存放一个地址值的内存单元，其中存储的是字符串所占用内存单元的首地址。 2. 赋值方式不同。对于字符数组，虽然可以在定义时初始化，但不能用赋值语句整体赋值。 3. 字符指针变量在定义后应先赋值才能引用。 4. 指针变量的值是可以改变的，字符指针变量也不例外；而数组名代表数组的首地址，是一个常量，而常量是不能改变的。
指针作为函数参数 ★★★★	1. 指针变量作为函数参数：指针变量既可以作为函数的形参，也可以作为函数的实参。指针变量作为函数参数，形参和实参之间的数据传递方式本质上是值传递，只是在调用函数时传递的内容是地址，这样使得形参变量和实参变量指向同一个变量。 2. 数组名作为函数参数：数组名作为函数形参时，接收实参数组的首地址；数组名作为函数实参时，将数组的首地址传递给形参数组。
指针型函数的定义 ★★	1. C 语言中，函数可以返回整型、实型、字符型数据，也可以返回指针类型数据，即返回一个地址。 2. 定义指针型函数的格式和有返回值的函数的定义格式基本相同，唯一的区别是在函数名前面加一个“*”，表示函数的返回值是指针型数据。
指针数组的定义 ★★	1. 指针数组是数组中的元素均为指针变量，可用于处理二维数组或多个字符串。指针数组的定义和一般数组的定义方法基本相同，但必须在数组名前面加“*”号。 2. 【格式】数据类型符 *指针数组名[数组长度]； 3. 【功能】定义指针数组，有“长度”个数组元素。

第八章 结构体类型和自定义类型

知识点名称	内容
结构体类型定义 ★★	1. 结构体类型定义的一般形式： <pre>struct 结构体类型名 /*struct 是结构体类型的关键字*/ { 数据类型符 成员名 1; 数据类型符 成员名 2; ... 数据类型符 成员名 n; };</pre> /*此行分号不能缺少*/ 2. 说明： (1) 数据类型相同的成员既可逐个、逐行分别定义，也可合并在一行定义。 (2) 结构体类型的定义允许嵌套，即结构体类型的成员既可以是基本数据类型，也允许是另一个已经定义的结构体类型。 (3) 对结构体类型变量的成员顺序地分配存储空间。
结构体类型变量 ★★★★	1. 定义结构体类型变量的一般格式： <pre>struct[结构体类型名] { }结构体类型变量表;</pre> 2. 结构体类型变量初始化的格式与一维数组相似：结构体类型变量={初值表};
结构体类型数组 ★★★★	1. 结构体类型数组定义的一般格式： <pre>struct 结构体类型名 { 成员列表; };</pre> struct 结构体类型名 结构体类型数组名[数组长度]; 2. 初始化的格式：结构体类型数组名[n]={初值表 1}, {初值表 2}, ..., {初值表 n}}; 3. 结构体类型数组元素成员的引用 (1) 结构体类型数组元素成员的引用：结构体类型数组名[下标].成员名 (2) 结构体类型数组元素成员地址的引用：&结构体类型数组名[下标].成员名
结构体类型指针 ★★★★★	1. 指向结构体类型变量的指针变量： (1) 定义：和定义结构体类型变量的方法基本相同，区别是在结构体类型指针变量名的前面加一个“*”。 (2) 如果指针变量 pointer 已指向结构体类型变量 var，则以下三种形式等价： 1) 方式一：var.成员 2) 方式二：pointer->成员 3) 方式三：(*pointer).成员 /* “*pointer” 外面的一对圆括号不能省略*/ 2. 指向结构体类型数组的指针变量 (1) 指针变量指向结构体类型数组元素 引用该元素的成员方法： 1) 方式一：(*指针变量).成员名

	2) 方式二：指针变量->成员名 (2) 指针变量指向结构体类型数组 引用下标为 i 的数组元素的成员方法： 1) 方式一：(* (指针变量+i)). 成员名 2) 方式二：(指针变量+i)->成员名
自定义类型 ★★★★	1. 基本数据类型符的自定义 【格式】typedef 基本数据类型符 用户类型符； 【功能】将“基本数据类型符”定义为“用户类型符”。 2. 数组类型的自定义 【格式】typedef 基本数据类型符 用户类型符[数组长度]； 【功能】以后可以使用“用户类型符”定义“基本数据类型符”的数组，其长度为定义时确定的数组长度。 3. 指针型的自定义 【格式】typedef struct <pre> { 数据类型符 成员名 1; 数据类型符 成员名 2; ... 数据类型符 成员名 n; } 用户类型符; </pre> 【功能】以后可以使用“用户类型符”定义含有 n 个成员的结构体类型变量、数组和指针变量等。

第九章 编译预处理

知识点名称	内容
编译预处理 ★★	1. C 语言提供的三种编译预处理命令分别是宏定义、条件编译和文件包含。 (1) 宏定义用于定义程序中的符号常量、类型别名，运算式代换和语句代换等。 (2) 文件包含是指一个源文件可以将另一个源文件的全部内容包含进来。 (3) C 语言提供了条件编译命令，使得用户可以选择对源程序的一部分内容进行编译，即不同的编译条件产生不同的目标程序。条件编译可以有效地提高程序的可移植性
宏定义命令 ★★★★	1. 无参宏定义 (1) 【格式】#define 宏名 字符序列 (2) 【说明】1) 宏定义不是 C 语言的语句，所以不能在行尾加分号。2) 对于双引号括起来的字符串内的字符，即使与宏名重名，也不进行宏展开。3) 宏定义命令放在文件的开头。如果要强制终止宏名的作用域，可以使用#undef 命令，灵活地控制宏定义的作用范围。 2. 有参宏定义 【格式】#define 宏名(形参表) 带参数的字符序列
文件包含命令 ★★★	1. 格式 【格式一】#include "文件名" 【格式二】#include <文件名>

	2. 说明 (1) 编译预处理时, 预处理程序将查找指定的被包含文件, 并将其复制到#include命令出现的位置上。 (2) 文件头部被包含的文件称为“标题文件”或“头部文件”, 常以“h”作为扩展名, 简称头文件。使用C语言提供的库函数编程时, 通常需要在源程序文件的开始部分包含相应的头文件, 例如若使用输入函数scanf()和输出函数printf(), 则需要文件包含命令#include<stdio.h>; 若使用取绝对值函数abs(), 则需要文件包含命令#include<math.h>。
条件编译命令 ★	1. 格式 (1) 【格式一】 <pre>#ifdef 标识符 程序段 1 #else 程序段 2] #endif</pre> 【功能】当“标识符”已经被#define命令定义, 则编译程序段 1; 否则编译程序段 2。其中[]中的内容, 即#else部分可以省略 (2) 【格式二】 <pre>#ifndef 标识符 程序段 1 [#else 程序段 2] #endif</pre> 【功能】当“标识符”没有被#define命令定义, 则编译程序段 1; 否则编译程序段 2。 (3) 【格式三】 <pre>#if 表达式 程序段 1 [#else 程序段 2] #endif</pre> 【功能】当表达式为“真”(非 0)时, 则编译程序段 1; 否则编译程序段 2。其中, []中的内容, 即#else部分可以省略。

第十章 文件

知识点名称	内容
文件分类 ★	1、按照文件的内容划分: 程序文件(又可以分为源文件、目标文件和可执行文件。)和数据文件。 2、按照文件的组织形式划分: 顺序存取文件和随机存取文件。 3、按照文件的存储形式划分: 可分为文本文件和二进制文件。 4、按照文件的存储介质划分: 可以分为普通文件即存储介质文件(磁盘、磁带等)和设备文件即非存储介质(键盘、显示器和打印机等)文件。
文件的打开和关闭函数 ★★★★★	1. 文件打开函数: FILE *fopen(char * filename, char *mode) (1) 文件使用方式中字符含义: r——读; w——写; a——追加; t——文本文件; b——二进制文件; +——读和写。

(2) 文件的使用方式说明:

文件类型	字符	使用方式	说 明
文本文件	"r"	只读	以只读方式打开一个已经存在的文本文件, 只允许读取数据, 不允许写入数据
	"r+"	读/写	以读/写方式打开一个已经存在的文本文件, 允许读取和写入数据
	"w"	只写	以只写方式打开一个文本文件, 只允许写入数据, 不允许读取数据。若文件已经存在, 则删去原文件的所有内容, 然后写入数据; 若文件不存在, 则自动创建一个新文件后写入数据
	"w+"	读/写	以读/写方式打开一个文本文件, 允许读取和写入数据。若文件已经存在, 则删去原文件的所有内容, 然后写入数据; 若文件不存在, 则自动创建一个新文件后写入数据
	"a"	追加	以追加的方式打开一个已经存在的文本文件, 只允许写入数据并追加到原文件尾
	"a+"	读/写	以读/写方式打开一个文本文件, 允许读取数据, 也允许写入数据到文件尾。若文件不存在, 则自动创建一个新文件
二进制文件	"rb"	只读	以只读方式打开一个已经存在的二进制文件, 只允许读取数据, 不允许写入数据
	"wb"	只写	以只写方式打开一个二进制文件, 只允许写入数据, 不允许读取数据。若文件已经存在, 则删去原文件的所有内容, 然后写入数据; 若文件不存在, 则自动创建一个新文件后写入数据
	"ab"	追加	以追加的方式打开一个已经存在的二进制文件, 只允许写入数据并追加到原文件尾
	"rb+"	读/写	以读/写方式打开一个已经存在的二进制文件, 允许读取和写入数据
	"wb+"	读/写	以读/写方式打开一个二进制文件, 允许读取和写入数据。若文件已经存在, 则删去原文件的所有内容, 然后写入数据; 若文件不存在, 则自动创建一个新文件后写入数据
	"ab+"	读/写	以读/写方式打开一个二进制文件, 允许读取数据, 也允许写入数据到文件尾。若文件不存在, 则自动创建一个新文件

2. 文件关闭函数: **int fclose(FILE *fp)**

(1) 【功能】关闭 fp 所指向的文件, 同时自动释放分配给该文件的内存缓冲区。

(2) 【返回值】若正确关闭指定的文件, 则返回 0; 否则返回非 0。

3. 标准设备文件的打开和关闭:

(1) stdin 指标准输入 (即键盘)

(2) stdout 指标准输出 (即显示器)

(3) stderr 指标准输入错误 (即显示器)

文件的
读/写函数

★★★

1. 文件尾测试函数: **int feof(FILE *fp)**

【功能】判断 fp 所指向的文件是否到达文件尾。

【返回值】若遇到文件尾, 则返回值是非 0; 否则, 返回值是 0。

2. 字符读(int fgetc(FILE *fp))/写函数 (int fputc(char ch, FILE *fp))

3. 字符串读(char *fgets(char *str, int length, FILE *fp))/写函数 (int fputs(char *str, FILE *fp))

4. 数据块读(fread())/写函数(fwrite())

5. 格式化读(fscanf())/写函数(sprintf())

文件的
定位函数

★★

1. 文件指针复位函数: **int rewind(FILE *fp)**

【功能】将 fp 所指向文件的位置指针重置到文件头, 即把文件的位置指针重新定位到 fp 所指向文件的起始位置

2. 文件随机定位函数: **int fseek(FILE *fp, long offset, int origin)**